

# Aggregation

**Aggregation → process + transform + analyze data**

Aggregation is like a pipeline where data goes in raw  
and comes out as useful insights.

## Use find() when:

- ☐ simple filtering
- ☐ simple sorting
- ☐ simple projection

is enough.

## Use Aggregation when:

Data must be:

- ☐ Transformed
- ☐ Reshaped
- ☐ Calculated
- ☐ Joined
- ☐ broken apart
- ☐ rebuilt

Imagine you need to find the total revenue  
generated by each product category in the  
last 24 hours.

# Aggregation Pipeline:

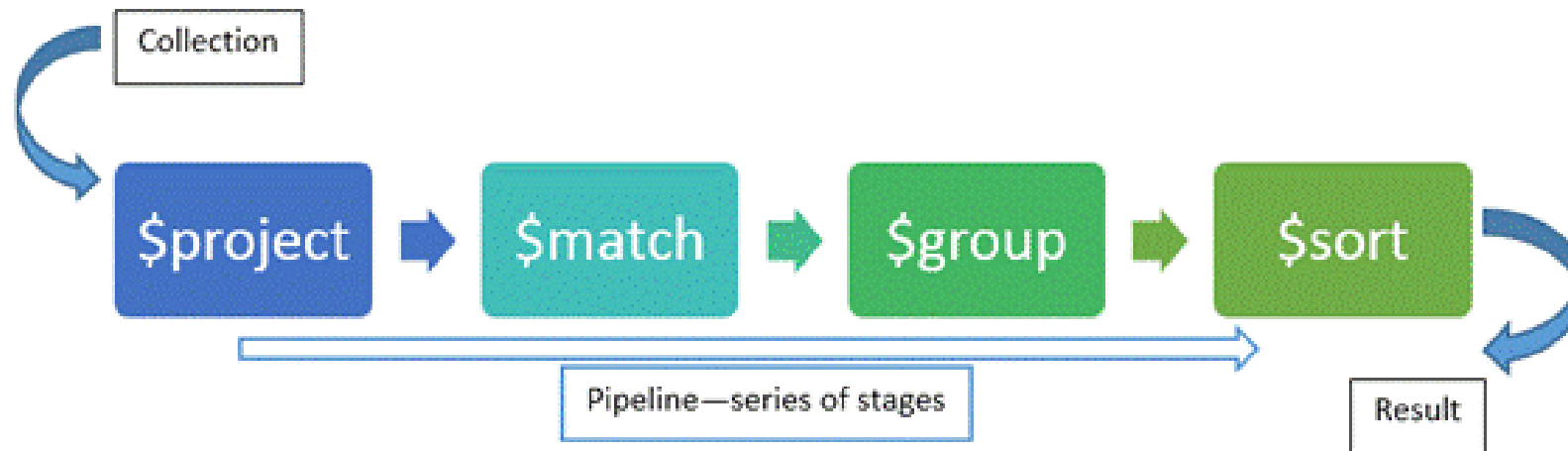
Collection → Stage1 → Stage2 → Stage3 → Result

## Syntax:

```
db.orders.aggregate([  
  { stage1 },  
  { stage2 },  
  { stage3 }  
])
```

### Concept:

- ❑ Each stage = one operation
- ❑ Output of one stage → input of next



# Pipeline Stages

These are the main steps in aggregation:

- ❑ \$match → filter data
- ❑ \$group → group data
- ❑ \$project → shape data
- ❑ \$sort → sort data
- ❑ \$lookup → join data
- ❑ \$unwind → break arrays

# \$match (Filter Data)

- ❑ Start with easiest
- ❑ Same as find()
- ❑ Always use early (performance)

Real example:

```
{ $match: { status: "delivered" } }
```

“Only delivered orders”

# \$group (Most Important)

- ❑ This is the heart of aggregation.
- ❑ The \$group stage is a fundamental part of the MongoDB Aggregation Pipeline used to categorize documents by a specific "group key" and perform calculations on the grouped data.
- ❑ It is conceptually similar to the GROUP BY clause in SQL.

```
db.sales.aggregate([
  {
    $group: {
      _id: "$category",           // Group by the category field
      totalRevenue: { $sum: "$price" },
      avgPrice: { $avg: "$price" },
      itemCount: { $sum: 1 }     // Count documents in each group
    }
  }
])
```

# \$project

\$project is used to:

- ☐ select fields
- ☐ remove fields
- ☐ rename fields
- ☐ create new fields
- ☐ transform data
- ☐ perform calculations

It works like: “Final output me kya-kya fields dikhani hain aur kaise dikhani hain.”

## Example:

```
db.students.aggregate([
  {
    $project: {
      _id: 0, // remove _id
      age: 1, // select fields
      studentName: "$name", // rename fields
      status: "Active", // create new field
      cityUpper: {
        $toUpper: "$city"
      }, // transform data
      totalMarks: {
        $add: ["$marks", 5]
      } // perform calculation
    }
  }
])
```

# \$sort

\$sort is used to sort documents in ascending or descending order.

```
db.students.aggregate([
  {
    $sort: {
      marks: -1,
      name: 1
    }
  }
])
```

- ❑ First sort by marks descending
- ❑ If marks same, then sort by name ascending.

# \$lookup

\$lookup is used to join data from another collection. It works like SQL JOIN.

```
{
  $lookup: {
    from: "collectionName", // Kis collection se join karna hai
    localField: "currentCollectionField", // Current collection ka field
    foreignField: "otherCollectionField", // Dusre collection ka matching field
    as: "outputArray" // Output kis naam se store hoga
  }
}
```

- ❑ First sort by marks descending
- ❑ If marks same, then sort by name ascending.

# \$unwind

\$unwind is used to break an array into separate documents.

```
db.users.aggregate([
  {
    $unwind: "$hobbies",
    preserveNullAndEmptyArrays: true // Empty array ko remove na kare
  }
])
```

- ☐ First sort by marks descending
- ☐ If marks same, then sort by name ascending.

**Thank  
You**